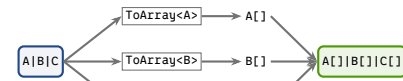


In one line: Type parameters are **solved by inference** at each call and bounded by extends. **Conditional, mapped** and **template-literal** types add a small pure functional language — if, pattern-bind, map over keys, recurse — that the checker runs at compile time and erases from the JavaScript, inside fixed depth and union-size limits.

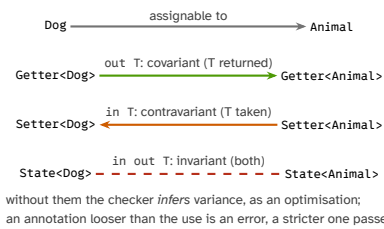
1 distributive conditional (2.8)

`ToArray<T> = T extends any ? T[] : never`

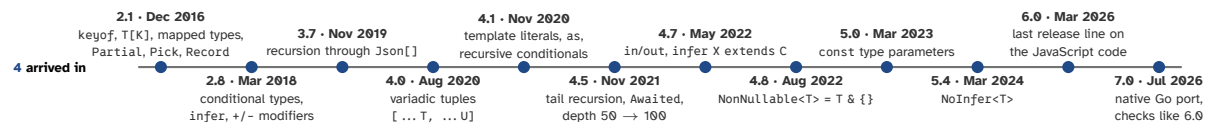
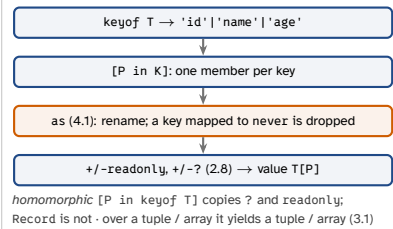


[T] extends [any]: no fan-out → (A|B|C)[]
only a *naked* type parameter distributes
never = the empty union → result never

2 variance: out / in (4.7)



3 a mapped type, step by step



Type parameters & inference

- Constraint** `<T extends HasId>`: an upper bound, met by shape; in the body only the constraint's members exist. `K extends keyof T` ties one parameter to another.
- Default** `<T = string>` (2.3): used when inference finds no candidate; it must satisfy the constraint; no required parameter after a defaulted one.
- Inference** gathers candidates from every argument: `combine<T>(a: T[], b: T[])` on `([1, 2, 3], ["hello"])` fails with *Type 'string' is not assignable to type 'number'* — pass `<string | number>` explicitly.
- Literals** are kept, not widened (`1 → number`), when the constraint includes a primitive or literal type; `f<T extends number>(x: T, y: T): T[]` makes `f(1, 2)` a `(1 | 2)[]`; without the constraint `number[]` (2.1, PR #10676).
- `<const T>` (5.0): infers as if the argument were written as `const (readonly ["a", "b"])`, but only for literals written in the call — a variable stays `string[]`. A mutable constraint gives a mutable tuple: `const T extends string[]` infers `["a", "b"]` (5.3, PR #55229; 5.0–5.2 fell back to `string[]`).
- `NoInfer<T>` (5.4, = `intrinsic`): the position is checked but yields no candidates — the other arguments decide `T`.
- Instantiation expression** (4.7): `const ErrorMap = Map<string, Error>` fixes the type arguments of a value; new `ErrorMap()` is a `Map<string, Error>`.
- Handbook rules: a type parameter should **appear twice**, or it relates nothing; **push it down** — `<T extends any[]>(arr: T)` returning `arr[0]` is inferred as `any`, `<T>(arr: T[])` as `T`; use as few as possible.

Operators on types

`keyof T`: the union of its keys — a string index signature gives `string | number` (`obj[0]` is `obj["0"]`), a number one `number`; `keyof any` = `string | number | symbol` (PropertyKey). `typeof v`: a value's type. `T[K]`: indexed access; `T[number]`: the element type of an array or tuple; a tuple declares `length` as a literal (`["string", number]["length"]` is 2). Variadic tuples (4.0): a spread may be generic and sit anywhere — `concat(a: T, b: U): [...T, ...U]` keeps both tuples' elements. 7.0 sorts types by content, so a printed union lists its members in one fixed order (6.0: `--stableTypeOrdering`).

Conditional types & infer (2.8)

- `T extends U ? X : Y` **resolves** to `Y` when the most permissive `T` is not assignable to `U`, to `X` when it definitely is — otherwise it is **deferred** and stays generic until `T` is known.
- `infer R` binds a piece of the match, and is allowed only in the extends clause. One name in several **covariant** spots → a union; in **contravariant** spots (parameters) → an intersection.
- Infering from an overloaded function uses its **last** signature only — for `ReturnType` too.
- `infer S extends string` (4.7) filters in place; 4.8 parses literals: `"100"` extends `'${infer N extends number}'` gives 100, but `"1.0"` gives `number` (it does not round-trip).

Traps

- Returning a new object where `T` was promised: `'length: number;'` is assignable to the constraint of type `'Type'`, but `'Type'` could be instantiated with a different subtype of constraint ... — the caller may hold a richer `T`.
- Omit/Pick on a union: `keyof (A | B)` has only the shared keys, so branch-only members vanish. `Omit` (3.5) does not distribute, by design (#31501): write `T extends any ? Omit<T, K> : never`.
- `T extends never ? ...` on `never` gives `never` (#23182): test `[T] extends [never]`.
- A return under a declared `T extends string ? A : B` does not narrow by the branch taken — each needs a cast. A fix was in the 5.8 beta (#56941), reverted (#61136); the retry (#61359) closed unmerged in Dec 2025.

Mapped & template literal types

- `keyof T` may hold number/symbol keys: remap via `Capitalize<string & K>`.
- Template literal types** (4.1) cross-multiply the unions in their holes: `'${"one" | "two" | "red" | "blue"} fish'` has 4 members. A union must stay below **100 000** members, else TS2590 *Expression produces a union type that is too complex to represent*.
- Matching** `S` extends `'...${infer X}...'`: a hole followed by literal text takes characters up to that text's **first** occurrence; a hole followed by another hole takes **one** character; the last takes the rest. 7.0 counts code points: an emoji is one character, not two surrogate halves.
- Uppercase, Lowercase, Capitalize, Uncapitalize are declared **intrinsic** in the lib: built into the compiler.

Recursion & limits

- 3.7 resolves type arguments lazily, so an alias may recurse through an array or generic: `type Json = string | number | boolean | null | Json[] | { [k: string]: Json }`. 4.1: conditional types may recurse — "responsibly and sparingly".
- 100** nested instantiations, or **5 000 000** in all, end in TS2589 *Type instantiation is excessively deep and possibly infinite*. Depth was 50 until 4.5 (PR #45711); not configurable; 7.0 keeps both (team, Sep 2026).
- Tail position** (4.5): a branch that is another conditional is evaluated in a loop, up to **1000** rounds. `Char | GetChars<Rest>` is not tail; an accumulator `Helper<Rest, Char | Acc>` is.
- `in/out` also speed up deeply recursive types: the checker compares type arguments instead of whole structures.

The lib (es5.d.ts), condensed

```
type Partial<T> = { [P in keyof T]?: T[P] };
type Required<T> = { [P in keyof T]-?: T[P] };
type Readonly<T> = { readonly [P in keyof T]: T[P] };
type Pick<T, K extends keyof T> = { [P in K]: T[P] };
type Record<K extends keyof any, T> = { [P in K]: T };
type Exclude<T, U> = T extends U ? never : T;
type Extract<T, U> = T extends U ? T : never;
type Omit<T, K extends keyof any> = Pick<T, Exclude<keyof T, K>>;
type NonNullable<T> = T & {}; // 4.8; was a conditional
type Parameters<T extends (...args: any) => any> =
  T extends (...args: infer P) => any ? P : never;
type ReturnType<T extends (...args: any) => any> =
  T extends (...args: any) => infer R ? R : any;
type Awaited<T> = T extends null | undefined ? T :
  T extends object & { then(onfulfilled: infer F, ...args: infer _): any }
  ? F extends ((value: infer V, ...args: infer _) => any) ? Awaited<V> : never
  : T;
type Uppercase<S extends string> = intrinsic; // NoInfer<T> too
```

Worked types

```
type Foo<T> = T extends { a: infer U; b: infer U } ? U : never;
type F1 = Foo<{ a: string; b: number }>; // string | number
type Bar<T> = T extends { f: (x: infer U) => void;
  g: (x: infer U) => void } ? U : never;
type B1 = Bar<{ f: (x: { a: 1 } => void;
  g: (x: { b: 2 } => void) }>; // { a: 1 } & { b: 2 }
type Handlers<E> = {
  [K in keyof E as `on${Capitalize<string & K>}`]: (e: E[K]) => void;
};
type H = Handlers<{ login: { id: string }; logout: undefined }>;
// { onLogin: (e: { id: string }) => void; onLogout: (e: undefined) => void }
type Params<S> = S extends `${string}:${infer P}/${infer R}`
  ? P | Params<`${R}`> : never;
type P1 = Params<`/u:/id/post/:pid`>; // "id" | "pid"
function lightC extends string<all: C[], start: NoInfer<C>>() {
  light(["red", "green", "blue"]); // error: Argument of type
  // "blue" is not assignable to parameter of type '"red" | "green"'.
```